

Problem Statement 2 – Smart Commuter Companion

1. Challenge Statement

Build a smart commuter companion mobile app that delivers proactive decision support during planned and unplanned events, tailored to commuters' needs.

Proactive – it reaches the commuter before the problem does, not after.

Decision support – it recommends an action, not just a status. "Take the 190 from Bt Batok, +11 min" beats "NSL delays".

Planned and unplanned – scheduled works, early closures and maintenance windows are known in advance and are just as disruptive; treat them as first-class, not an afterthought.

Tailored – the same disruption means different things to different people. See **2.2 The Commuter**.

2. Challenge Details

2.1 Introduction

Singapore's transport network works well on an ordinary day. The commuter's problem is the day that is not ordinary – a signalling fault at 08:15, a station exit closed for works, a line running at reduced frequency, a downpour that turns a 6-minute walk into a decision.

Today the burden of reacting falls on the commuter: notice something is wrong, work out whether it affects them, decide what to do instead, and do it while standing on a platform. Most apps are reactive and generic – they tell everyone the same thing, after the fact.

We want the opposite. An app that knows enough about *this* commuter's routine to tell them, before they leave, that today is different and here is what to do about it.

2.2 The Commuter

The user here is the commuter, not an operator. Three personas define what that means in practice – design against them.

You do not need to serve all three well – but say which one you are building for, and show a real journey working end to end for that person.

1. Rachel – the fixed-schedule commuter.

Tampines to Raffles Place, EWL, leaves 07:40, must be at her desk by 08:45. Has done the same trip for four years and does not check any app on a normal day. A five-minute delay is noise; a fifteen-minute delay costs her a meeting. She needs to be interrupted *only* when it matters, and told what to do in one line.

2. Arjun – the multi-modal, flexible-start worker.

Punggol to one-north. Cycles to the LRT, sometimes takes a bus the whole way if the weather is good, start time flexible within about an hour. Optimises for comfort and predictability over pure speed, and will happily leave twenty minutes later to avoid a crush. Cares about crowding, sheltered routes and whether he can bring his bike.

3. Mdm Lim – the accessibility-constrained occasional traveller.

Bedok to Singapore General Hospital for a fortnightly appointment. Walks slowly, avoids stairs, needs lifts and sheltered walkways, and will not improvise a reroute on the platform. For her the app must be usable in large text, must plan the whole trip in advance including the walk at each end, and must warn her the day before if a lift or exit is out of service.

You may propose your own persona if you can justify it, but do not silently build for "a generic commuter" – that is how apps end up serving nobody.

2.3 The Geospatial Base (OpenStreetMap)

OpenStreetMap is the required geospatial base for this problem statement – see **3.2.2** for the requirement itself. This section covers where to obtain it and the two conditions attached to its use.

Where to get it:

Need	Option
Bulk Singapore data	Geofabrik extract – https://download.geofabrik.de/asia/malaysia-singapore-brunei.html
Targeted queries	Overpass API – https://overpass-api.de (see https://overpass-turbo.eu to build queries interactively)
Routing engines	OSRM (https://project-osrm.org), GraphHopper, or Valhalla – all open source and self-hostable, all consume OSM data
Map rendering	Leaflet (https://leafletjs.com) or MapLibre GL JS (https://maplibre.org)

Two conditions on OSM use:

- **Attribution is required.** OSM data is licensed under the ODbL. Display "© OpenStreetMap contributors" wherever you show the map or derived data.
- **Do not hammer the public tile servers.** `tile.openstreetmap.org` is run on donated infrastructure and its usage policy (<https://operations.osmfoundation.org/policies/tiles/>) prohibits heavy or bulk use by applications. For a demo, use a free tile provider's own key, self-host tiles, or bundle a static extract. The same applies to the public Overpass instance – cache what you fetch rather than querying it in a loop.

OSM is the required base, not an exclusive one. Layer LTA's DataMall geospatial layers on top of it where they are better – `CoveredLinkWay`, `TrainStationExit` and `CyclingPath` are authoritative in a way crowd-sourced data is not. OneMap's routing API is also fair game alongside your own.

2.4 The Data

Provided in this repository

- `PS2/data/AmendmenttoMP2014RailStation.geojson` – rail station geometry: 208 polygons. Note these are **station footprints, not points**; `TYPE` carries only MRT/LRT/CCL, not line codes; and `crs` is null, so confirm the coordinate reference system before you join it to anything. A `GRND_LEVEL` attribute is present and may be useful for the accessibility persona.
- `PS2/data/UsefulWebsites.txt` – every external source named in this section, with its URL and what it is for, in one place.
- `PS2/references/LTA_DataMall_API_User_Guide.pdf` – full DataMall API documentation, version 6.8.
- `PS2/references/24hourWeatherForecast.json`, `PS2/references/4dayWeatherForecast.json` – OpenAPI specifications for two of the data.gov.sg weather endpoints. These are **interface documents, not weather data** – they describe the request parameters and response shape so you can write against the API without guessing.

LTA DataMall – <https://datamall.lta.gov.sg>

Register for a free AccountKey, then call

`https://datamall2.mytransport.sg/ltaodataservice/<endpoint>` with an `AccountKey` header. Responses page 500 rows at a time via `$skip`.

The endpoints most relevant to this problem statement:

Endpoint	What it gives you	Refresh
<code>TrainServiceAlerts</code>	Official structured train disruption feed — see below, this is the single most important endpoint for this brief	Ad hoc
<code>PCDRealTime</code>	Station Crowd Density (Real-Time) — crowdedness per station, by line	10 min
<code>PCDForecast</code>	Station Crowd Density (Forecast) — per station, at 30-minute intervals	24 h
<code>v3/BusArrival</code>	Bus ETA at a stop, plus the <code>Load</code> field — how crowded each arriving bus is	Real-time
<code>BusServices</code> / <code>BusRoutes</code> / <code>BusStops</code>	Network reference data	Ad hoc
<code>EstTravelTimes</code>	Estimated travel times on expressway segments	5 min
<code>v4/TrafficSpeedBands</code>	Road speed bands — a congestion proxy for bus journeys	5 min
<code>TrafficIncidents</code>	Live road incidents	2 min
<code>RoadWorks</code> / <code>RoadOpenings</code>	Approved road works and planned road openings — the "planned event" half of the brief	Ad hoc
<code>PlannedBusRoutes</code>	New and changed bus routes published <i>in advance</i> of their effective date	Ad hoc
<code>VMS</code>	Traffic advisories currently displayed on EMAS signboards along expressways and arterial roads	2 min
<code>v2/FacilitiesMaintenance</code>	Ad hoc lift maintenance at MRT stations, down to the individual lift and which exit it serves — essential for the accessibility persona	Ad hoc
<code>PubFloodAlerts</code>	Public flood alerts	Ad hoc
<code>Taxi-Availability</code> / <code>TaxiStands</code>	Taxi supply and stand locations	1 min

Endpoint	What it gives you	Refresh
<code>CarParkAvailabilityv2</code>	Carpark lot availability	1 min
<code>BicycleParkingv2</code>	Bicycle parking near a location	Ad hoc
<code>PV/Train</code> , <code>PV/Bus</code> , <code>PV/ODTrain</code> , <code>PV/ODBus</code>	Historical passenger volume, including origin-destination pairs	Monthly
<code>GeospatialWholeIsland?ID=<layer></code>	Geospatial layers as SHP files (see below)	Ad hoc

Train Service Alerts – read this endpoint before you design anything

`GET /ltaodataservice/TrainServiceAlerts` is the official structured feed for train service unavailability, and it carries considerably more than a status flag:

The response has three top-level parts, and the per-disruption detail is **nested inside `AffectedSegments`** rather than sitting flat – a shape worth checking against a live response before you write the parser:

Field	Value
<code>Status</code>	<code>1</code> normal service or minor delays · <code>2</code> disrupted service or major delays
<code>AffectedSegments</code>	A list. One entry per affected segment – empty on a normal day
<code>Message</code>	A list , separate from <code>AffectedSegments</code> . Each entry carries <code>Content</code> (the advisory text) and <code>CreatedDate</code>

Each `AffectedSegments` entry carries:

Field	Value
<code>Line</code>	Affected line code
<code>Direction</code>	<code>Both</code> , or <i>(towards station name)</i>
<code>Stations</code>	Affected station codes, e.g. <code>NE1</code> , <code>NE3</code> , <code>NE4</code> , <code>NE5</code> , <code>NE6</code>

Field	Value
FreePublicBus	Stations where free boarding onto normal public buses is available – or Free bus service island wide
FreeMRTShuttle	Stations served by free MRT shuttle services , with the shuttle's own route encoded
MRTShuttleDirection	Both , or (towards station name)

Note what this means: **the mitigation is in the feed**. You do not have to infer which buses might help – when LTA activates free bus boarding or a shuttle, the endpoint tells you where. The interesting work is not discovering that a disruption exists; it is deciding what *this* commuter should do about it, and telling them at the right moment.

A trap: line codes are not consistent across endpoints

The same physical line has different codes depending on which API you call. This will bite you when you join data.

Line	TrainServiceAlerts	Station Crowd Density
Sengkang LRT	STL	SLRT
Punggol LRT	PTL	PLRT
Bukit Panjang LRT	BPL	BPL
Circle Line Extension	folded into CCL	separate code CEL
Changi Extension	folded into EWL	separate code CGL

Build one canonical line table early and map everything through it.

Crowding – three separate signals, not one

Crowding is central to this problem statement, and DataMall exposes it in three different places. Teams that only find one of them will under-serve the personas.

1. Station crowd density, real-time – `GET /ltaodataservice/PCDRealTime?TrainLine=<code>`

Crowdedness of every station on one line, refreshed every 10 minutes. Response carries `Station`

(station code, e.g. `EW13`), `StartTime`, `EndTime`, and `CrowdLevel` with values `l` (low), `m` (moderate), `h` (high) or `NA`. One call per line — pass `TrainLine` as `NSL`, `EWL`, `CGL`, `CCL`, `CEL`, `NEL`, `DTL`, `BPL`, `SLRT`, `PLRT` or `TEL`.

2. Station crowd density, forecast — `GET /ltaodataservice/PCDForecast?TrainLine=<code>`

The same `CrowdLevel` scale, but forecast in **30-minute intervals**, published once a day. This is the endpoint that makes *proactive* advice possible: it lets the app tell a commuter at 07:30 that their usual 08:15 platform will be `h`, and suggest leaving earlier or taking a different route — before anything has gone wrong.

3. Bus occupancy — the `Load` field in `GET /ltaodataservice/v3/BusArrival?BusStopCode=<code>`

Per arriving bus, not per stop: `SEA` (seats available), `SDA` (standing available), `LSD` (limited standing). The same response also carries `Feature` = `WAB` for wheelchair-accessible vehicles and `Type` = `SD` / `DD` / `BD` for single-deck, double-deck or bendy — both directly relevant to the accessibility persona, and to anyone deciding whether the next bus is worth boarding or worth waiting for.

Historical baseline — `PV/Train`, `PV/Bus`, `PV/ODTrain`, `PV/ODBus` give monthly passenger volumes including origin-destination pairs. Use these to learn what "normal" looks like for a given station and hour, so the app can tell an unusual crowd from a Tuesday.

Naming note: these two APIs were originally called **Platform** Crowd Density and were renamed to **Station** Crowd Density. The URL paths (`PCDRealTime`, `PCDForecast`) were not changed. TEL is now supported on both.

Geospatial Whole Island layers (ANNEX E of the guide) — pass the layer ID to `GeospatialWholeIsland`. The ones worth your attention:

- `CoveredLinkWay` — **sheltered walkways**
- `CyclingPath` — cycling network
- `Footpath` — pedestrian paths
- `PedestrianOverheadbridge_UnderPass` — overhead bridges and underpasses (*the spelling is LTA's, not a typo on our part — use it verbatim*)
- `TrainStation`, `TrainStationExit` — station and **exit** locations; exits matter more than stations for door-to-door routing
- `BusStopLocation`, `TaxiStand`

Service notices in free text — <https://t.me/s/sgmrt>

`TrainServiceAlerts` gives you today's notices, structured. What it cannot give you is *history*: on an ordinary day `AffectedSegments` is empty, so over a hackathon weekend you will collect very few real disruption examples from it.

The **SG MRT Updates** Telegram channel mirrors SMRT and SBS Transit announcements as they are posted and keeps them, which makes it an archive of how these notices are actually worded – useful if you want to build or evaluate anything that reads a service notice and turns it into advice, and the only practical way to assemble a set of examples the app has not already seen.

Read it as reference material, not as a feed. Two caveats: the same notice is often posted several times with small wording changes as it is updated, and the web view shows times but not dates.

data.gov.sg – <https://data.gov.sg>

Open government data, free, no key required for the real-time weather APIs:

- 2-hour nowcast – <https://api-open.data.gov.sg/v2/real-time/api/two-hr-forecast>
- 24-hour forecast – <https://api-open.data.gov.sg/v2/real-time/api/twenty-four-hr-forecast>
- 4-day outlook – <https://api-open.data.gov.sg/v2/real-time/api/four-day-outlook>
- Rainfall readings by weather station – <https://api-open.data.gov.sg/v2/real-time/api/rainfall>
- Air temperature, humidity, wind and PM2.5 are available on the same [/v2/real-time/api/](https://api-open.data.gov.sg/v2/real-time/api/) base.

Browse the wider catalogue at <https://data.gov.sg/datasets>. Things worth searching for, because they change how and when people travel:

- **Public holidays** – a Monday holiday reshapes the whole week's demand
- **School terms and holidays** – morning peaks shift noticeably outside term time
- **HDB and population data** – for reasoning about origin-side demand
- **Historical ridership** – complements DataMall's `PV/*` endpoints

Other government sources worth a look

- **NEA** – weather, air quality and heat data feed the walking and cycling legs of a journey.
- **Singapore Government Design System** (<https://designsystem.tech.gov.sg>) – if you want the app to look and behave like a Singapore government service.
- **Health facility and accessibility datasets** on data.gov.sg – relevant if you build for the accessibility persona.

Singapore's official mapping service. Free with registration. Useful for geocoding, reverse geocoding, and its **routing API**, which supports walk, drive, cycle and public-transport modes – worth looking at before you build your own routing engine.

2.5 Ground Rules on Data Use

Other data – encouraged, with one condition

We actively encourage you to bring in data we have not listed. A commuter companion is exactly the kind of product that gets better when it knows about school terms, public holidays, stadium events, weather, or anything else that changes how people travel. Some of the most interesting entries will be the ones that use a source nobody else thought of.

The condition is that you have the right to use it. Concretely:

- **Official APIs and open-data portals are always safe.** LTA DataMall, data.gov.sg, OneMap, and any public API with published terms you comply with.
- **Read the terms of service** for anything else, and keep to any rate limits and attribution requirements. Note the source and its licence in your submission.
- **Scraping is generally not acceptable** for this problem statement. Most sites prohibit it in their terms, it is fragile, and it can breach the platform's rules even where the content is publicly visible. If an official API exists, use the API. If you believe a specific scrape is both permitted by that site's terms and necessary, raise it with us before you build on it – do not assume.
- **Do not use personal data** you have not been given permission to use. If your app collects a user's routine or location, say in your submission what you store, where, and for how long. A commuter companion is a privacy-sensitive product by nature; treat it as one.
- **Do not commit credentials.** API keys go in environment variables or an ignored `.env`, never in the repository.

If in doubt about a source, ask us. We would rather answer a question than disqualify a good idea.

2.6 Additional Notes

Underground, there is no signal. A commuter between stations cannot fetch anything. Decide what your app does in that window – cache the current journey, degrade gracefully, or say plainly that it is stale – and state the choice in your write-up.

The feeds are quiet most days. `TrainServiceAlerts.AffectedSegments` is empty on a normal day, so do not build a demo that depends on a real disruption occurring during judging. You may demonstrate the major-disruption path by replay or injected test data **provided it is labelled as such**.

Everything else in the feed — including its **Message** stream of live bus diversions and advisories — is populated daily and needs no such allowance.

Ask if you are unsure. Especially about a data source you are not certain is in bounds — see **2.5 Ground Rules on Data Use**, and ask rather than guess. We would rather answer a question than disqualify a good idea.

3. Expectations & Goals

3.1 Scope

Build a web app, designed mobile-first. No native build, no app store — a web application a commuter opens in their phone's browser.

Mobile-first means the phone is the primary target, not an afterthought you shrink down to. Judges will open your app **in a mobile browser on a real phone**, not in a resized desktop window, and score the experience there. If it also works well on a desktop, good — but a desktop layout squeezed onto a phone will not score.

Design for the real conditions: a small screen, one thumb, a moving train, bright sunlight, an address bar that appears and disappears as you scroll, a soft keyboard that eats half the viewport, and — underground — no signal at all.

What we are *not* asking for is a dashboard for operators. The user here is the commuter.

3.2 Required Capabilities & Judging Rubric

Three things are **mandatory**. A submission missing any of them is incomplete, however good the rest is.

3.2.1 Route planning

The app must plan an actual journey, not just report status. Given an origin, a destination and a time, it must produce a route the commuter can follow — and when conditions change, it must produce a **revised** one.

At minimum:

- Multi-modal where the persona needs it (rail, bus, walk, and cycle for Arjun)
- Door to door, including the walking legs at each end — a route that starts at a station and ends at a station is not a commuter's journey
- Responsive to live conditions: a disruption, a crowded platform or heavy rain should change what the app recommends, and the app should say *why* it changed

- Realistic timing, with the uncertainty made visible rather than hidden behind a single confident number

You may build routing on top of an existing engine – see **2.3 The Geospatial Base**. We are not asking you to invent a shortest-path algorithm; we are asking for routing that reflects what is actually happening on the network today.

3.2.2 GIS on OpenStreetMap

Use OpenStreetMap as your geospatial base. It is open, it is licensed for this use, and it carries the pedestrian and cycling detail this problem needs – footways, crossings, stairs, lifts, covered walkways, cycle paths – that a plain road map does not.

See **2.3 The Geospatial Base** for where to obtain OSM data, the attribution requirement, and the tile-server usage policy.

3.2.3 Visualisation

The commuter must be able to see the situation, not just read about it. The app needs a visual layer that makes the state of their journey legible at a glance on a phone screen.

What we expect to see handled:

- The route itself on a map, with the affected portion clearly distinguished from the unaffected portion
- The alternative shown against the original, so the commuter can judge the trade-off rather than being told to trust the app
- Crowding rendered in a way that is readable in one second – this is a three-level scale, it does not need a chart
- Time and delay shown so the cost of each option is obvious

Two things we will be looking for specifically:

- **It works on a phone, in motion, in one hand.** Small screen, bright sunlight, moving train, possibly one thumb. Test it on an actual phone browser – mobile emulation in devtools hides viewport-height, soft-keyboard and touch-target problems that only show up on the real thing.
- **Every visual element earns its place.** A map that shows dots with no context, or a chart nobody would act on, is worse than not having it – it costs screen space and attention that a commuter standing on a platform does not have.

Accessibility is part of this, not separate from it: legible type sizes, sufficient contrast, and not relying on colour alone to convey a state.

3.2.4 Judging Rubric

Submissions are graded against three criteria, weighted to **100%**:

Criterion	Weight	What Judges Look For
Problem Fit	40%	Would a real commuter be better off with this app than without it? Persona fit, proactivity and the quality of the decision it offers – plus whether the submission does anything beyond what this brief asked for.
Technical Execution	35%	Routing that reflects live conditions on an OpenStreetMap base, the breadth and judgement of the data brought in, and whether the thing is actually built – runs from a clean README, performance is sane.
Ease of Use	25%	A commuter could pick it up on a phone, one-handed, and get an answer. Interaction design, information hierarchy and accessibility. Scored on a real phone browser, not desktop emulation.

Each criterion is scored on a 0–5 scale and weighted. **Commuter value and design together carry more than half the total** – a technically impressive app that a commuter would not open twice cannot score well here, and that is deliberate.

Mandatory-capability cap. A submission missing one of the three required capabilities in 3.2.1–3.2.3 cannot exceed level 3 on the part of the score that covers it.

Caps apply regardless of other merit. A feature shown in the pitch but absent from the running system, mocked data presented as live, a claim a judge cannot verify, a credential committed to the repository, or OpenStreetMap used without attribution – each caps the part of the score it touches, and the attribution one is a licence breach rather than a style point.

How judging runs. Judges follow your README on a clean machine; the app is opened in a browser on a real phone; you walk one real journey for your chosen persona end to end; and you are asked to defend one claim per criterion. Saying it is in the slides is not evidence.

3.3 Beyond the Brief

Part of **Problem Fit** is whether a submission does anything beyond what this brief asked for. This section is where that is earned. It describes the problem as we currently understand it; a submission that surfaces a capability we did not ask for, and that on seeing it we would want, scores at the top of it.

None of it is required. A submission that answers the brief exactly and claims nothing beyond it can still score well.

3.3.1 Use of AI

Using AI is not a requirement – but where you use it, it must earn its place.

We expect most teams to use AI while building. That is fine and expected. What scores is AI that makes the *product* better for the commuter, not AI used for its own sake. Some directions that fit this problem naturally:

- Turning free-text service notices into structured, personalised advice
- Predicting disruption duration, or the probability a delay escalates, so the app can advise "wait" versus "reroute"
- Learning an individual's routine from their travel history and inferring which disruptions actually matter to them
- Conversational or voice interaction for hands-free use on a platform
- Ranking alternatives against a persona's real preferences rather than shortest-time-wins

An AI feature you can show works — with a measurement, however simple — is worth far more than one you can only describe. A well-argued decision *not* to use a model where a simpler method works is also creditable.

GCP credits are expected to be available to participating teams. Details will be confirmed separately. If you plan to depend on a specific GCP service, tell us early so we can make sure access is in place.

Whatever you use, a judge must be able to check it without paying for anything. If something can only be demonstrated or regenerated by spending money, it does not count.

3.3.2 Open Innovation

Anything that delivers real value beyond what 3.2 asks for. We are not prescribing a list — if we could, it would be in 3.2.

To be creditable, an idea must be sound enough to survive scrutiny: stated assumptions, known limits, and a plausible path to production. An original idea nobody can use is not yet a contribution — Problem Fit is led by whether a real commuter is better off, and originality is judged in that light.

4. Deliverables

How to package and hand these in is covered in

[PS2/submission/README.md](#) — read it before you submit.

1. **The app** — runnable, with clear setup instructions, covering all three required capabilities in **3.2**. Judges will follow your README on a clean machine; what does not run does not score.
2. **A short write-up** — which persona you built for, your architecture, your assumptions, and the limitations you know about. If you claim a number anywhere — an accuracy figure, a speed-up, a comparison against an alternative — say how you arrived at it. A claim a judge cannot check does not score.
3. **A demo** — walk one real commuter journey, end to end, through one real disruption.

Questions. Ask the organisers – especially about a data source you are unsure of. See **2.5 Ground Rules on Data Use**.